

	Naive	Divide and Conquer	Strassen
64 x 64	.004 seconds	.003 seconds	.005 seconds
128 x 128	.014 seconds	.011 seconds	.012 seconds
256 x 256	.084 seconds	.074 seconds	.063 seconds
512 x 512	.615 seconds	.551 seconds	.405 seconds
1024 x 1024	4.864 seconds	4.337 seconds	2.749 seconds
2048 x 2048	44.513 seconds	37.333 seconds	19.192 seconds

In the smaller 64 x 64 matrix multiplication the naive and the divide and conquer algorithm is faster than Strassen's algorithm due to the large overhead in Strassen's with allocating memory for the sub matrices in each recurrence loop. This leads to the sub matrices in Strassen's once reaching a threshold of $n = 32$ it calls the naive algorithm to solve it otherwise, it takes exponentially longer due to the memory management. The same was done for the divide and conquer algorithm as it too was slower than the naive algorithm otherwise.

Once the dimensions of $n = 128$ the Strassen's algorithm begins to match the naive and the divide and conquer algorithms in time. This is due to the closing of the gap of the amount of arithmetic computations needed for the matrix multiplication. At dimensions $n = 256, 1024$ and 2048 the gap begins to widen in Strassen's algorithm favor. This is due to the fact that Strassen's requires one less matrix to multiply at each level of recursion and the higher the dimension the more levels of recursion there are.